# Lekcja 9. Wprowadzenie do programowania obiektowego

Klasa 4 technik programista | Programowanie aplikacji | INF.04

Notatka obejmuje: paradygmaty programowania, programowanie imperatywne, proceduralne i strukturalne oraz pierwsze pojęcia programowania obiektowego: klasa, obiekt i cechy OOP.

## 1. Skróty i pojęcia

| Pojęcie | Wyjaśnienie |
|---|---|
| INF.04 | kwalifikacja obejmująca projektowanie, programowanie i testowanie aplikacji. |
| OOP / POO | programowanie zorientowane obiektowo, czyli sposób projektowania programu z użyciem klas i obiektów. |
| main() | funkcja główna programu w językach C/C++, od której zwykle rozpoczyna się wykonywanie kodu. |
| goto | instrukcja skoku; w kodzie strukturalnym unika się jej, ponieważ zaburza naturalny przepływ programu. |
| top-down | projektowanie „z góry na dół”: najpierw problem ogólny, potem coraz bardziej szczegółowe podproblemy. |
| bottom-up | projektowanie „z dołu do góry”: najpierw małe elementy, potem budowanie z nich większego systemu. |

## 2. Paradygmat programowania

Paradygmat programowania to sposób myślenia o rozwiązaniu problemu oraz organizacji programu. Nie jest to konkretny język programowania, lecz styl projektowania kodu i dobierania technik programistycznych.

- Programowanie imperatywne opisuje, jak krok po kroku wykonać zadanie.
- Programowanie deklaratywne skupia się bardziej na tym, co ma być wynikiem.
- Programowanie proceduralne dzieli problem na funkcje/procedury.
- Programowanie obiektowe wiąże dane z operacjami wykonywanymi na tych danych.

## 3. Programowanie imperatywne

W programowaniu imperatywnym program jest ciągiem instrukcji wykonywanych kolejno. Instrukcje operują na danych zapisanych w zmiennych. Konkretne wartości zmiennych tworzą stan programu, który zmienia się podczas działania programu.

### Przykład idei:

```
int licznik = 0; // stan programu
licznik = licznik + 1; // zmiana stanu
if (licznik > 10) {
    cout << "koniec";
}
```

W starszych programach imperatywnych występowała instrukcja skoku goto. Współcześnie należy jej unikać, ponieważ utrudnia analizę, testowanie i modyfikację programu.

## 4. Programowanie proceduralne

Programowanie proceduralne polega na dzieleniu problemu na podprogramy, czyli procedury lub funkcje. Funkcja rozwiązuje zadanie cząstkowe i komunikuje się z otoczeniem za pomocą argumentów oraz wartości zwracanej.

- W C++ odpowiednikiem procedury może być funkcja typu void.
- Funkcje zdefiniowane przez programistę mogą znajdować się w tym samym pliku co program główny albo w osobnych plikach/bibliotekach.
- Zaleca się, aby funkcje nie modyfikowały niepotrzebnie przekazanych argumentów.

Nie należy mylić programowania proceduralnego z programowaniem funkcyjnym. Programowanie funkcyjne jest osobnym paradygmatem, powiązanym z modelem deklaratywnym.

## 5. Podejście top-down i dekompozycja

W podejściu „z góry na dół” najpierw formułuje się rozwiązanie ogólne, a następnie dzieli je na mniejsze podproblemy. Proces podziału problemu na mniejsze części nazywa się dekompozycją.

### Schemat przykładowej dekompozycji:

```
Problem główny
obsłuż zamówienie
        →
Podproblemy
wczytaj dane
oblicz koszt
wydrukuj wynik
```

## 6. Programowanie strukturalne

Programowanie strukturalne wykorzystuje struktury sterujące, które porządkują wykonanie programu. Podstawowe struktury to:

- sekwencja - instrukcje wykonywane po kolei;
- wybór - instrukcje warunkowe, np. if, if-else, switch;
- iteracja - pętle, np. while, do-while, for.

W programowaniu strukturalnym struktury można zagnieżdżać. Należy unikać niekontrolowanych skoków, ponieważ niszczą uporządkowany przepływ sterowania. Każda struktura sterująca może być traktowana jak blok z jednym wejściem i jednym wyjściem.

Relacja pojęć: każdy program proceduralny powinien być zgodny z zasadami programowania strukturalnego, ale nie każdy program strukturalny musi korzystać z funkcji.

# 7. Programowanie zorientowane obiektowo

Programowanie zorientowane obiektowo jest kolejnym etapem rozwoju metod programowania. Jego główną ideą jest ścisłe powiązanie danych z funkcjami, które na tych danych operują. Program traktuje się jako zbiór obiektów, które mają własny stan i określone zachowania.

W programowaniu proceduralnym funkcje i dane są często traktowane osobno. W programowaniu obiektowym umieszcza się je we wspólnych konstrukcjach, pełniących rolę kontenerów.

# 8. Klasa i obiekt

Klasa to podstawowy element konstrukcyjny programowania obiektowego. Jest złożonym typem danych definiowanym przez programistę. Klasa opisuje, jakie dane i jakie operacje będą dostępne dla obiektów tej klasy.

Obiekt to instancja klasy. Powstaje w pamięci operacyjnej podczas działania programu. Obiekt posiada własny stan, który jest przechowywany w jego zmiennych członkowskich, oraz zachowanie opisane przez metody.

## Schemat pojęciowy:

| | | |
|---|---|---|
| Klasa<br>wzorzec: pola + metody | → tworzy → | Obiekt<br>konkretna instancja klasy |

## Prosty zapis idei w C++:

```
class Ksiazka {
public:
    string tytul;
    void wyswietl();
};

Ksiazka lektura; // obiekt klasy Ksiazka
```

# 9. Podstawowe cechy OOP

- Hermetyzacja / enkapsulacja - łączenie danych z metodami i ukrywanie szczegółów działania obiektu.
- Dziedziczenie - możliwość tworzenia nowych klas na podstawie klas już istniejących.
- Polimorfizm - możliwość różnego zachowania obiektów przy tym samym typie wywołania.
- Abstrakcja - skupienie się na cechach istotnych i pominięcie niepotrzebnych szczegółów.

# 10. Do zapamiętania

- Paradygmat to styl rozwiązywania problemów i organizacji kodu.
- Program imperatywny zmienia stan programu przez wykonywanie instrukcji.
- Program proceduralny dzieli problem na funkcje lub procedury.

- Program strukturalny korzysta z sekwencji, wyboru i iteracji oraz unika niekontrolowanych skoków.
- Program obiektowy organizuje kod wokół klas i obiektów.